Interview Preparation Guide

Anushka Arora · Cloud & DevOps Engineer | AI Engineer

A question bank for every skill domain and project on the resume, organised from fundamentals to advanced. Levels: **Basic** (definitions and core concepts), **Intermediate** (how things work and trade-offs), **Advanced** (design, scale, failure modes, and real decisions). Treat each as something a senior interviewer could drill into — be ready to explain the *why*, not just the *what*.

Part 1 — Skill Domains

AWS Cloud & Infrastructure

What they're testing: *Tests whether you understand the building blocks and can design secure, available systems.*

BASIC

1. What is the difference between a Region and an Availability Zone?
2. Explain public vs private subnets and how traffic reaches each.
3. Differentiate an IAM user, an IAM role, and an IAM policy. When do you use a role over a user?
4. What is the difference between a Security Group and a Network ACL?
5. What are the main EC2 instance families, and when would you choose compute-optimised vs memory-optimised?
6. What are the S3 storage classes and what drives the choice between them?

INTERMEDIATE

1. How does an Elastic IP differ from an auto-assigned public IP, and when are you billed for one?
2. Walk through how a Site-to-Site VPN is established. What happens in IKE Phase 1 vs Phase 2, and what is DPD?
3. Compare ALB, NLB, and Gateway Load Balancer. Give a use case for each.
4. How does ACM issue and auto-renew certificates, and how do you attach one to an ALB listener?
5. What is SSM Session Manager and why is it often preferred over opening SSH/RDP on a bastion?
6. Explain VPC endpoints (Gateway vs Interface/PrivateLink) and the cost/security benefit.

ADVANCED

1. Design a highly available 3-tier application across multiple AZs. Where does each tier live and how does traffic flow?
2. How would you give one account's role secure, least-privilege access to another account's resources using STS AssumeRole?
3. Compare VPC Peering, Transit Gateway, and PrivateLink. When does each win?
4. A private subnet needs outbound internet for patching but no inbound. How do you design egress (NAT GW, egress-only IGW, endpoints) and control cost?
5. How do you think about data-transfer cost in a design (intra-AZ, cross-AZ, internet egress, NAT processing)?
6. How would you detect and respond to a leaked IAM access key programmatically?

Kubernetes

What they're testing: *Tests workload modelling, networking, and what you do when a cluster misbehaves.*

BASIC

1. What is the difference between a Pod, a ReplicaSet, and a Deployment?
2. When do you use a StatefulSet instead of a Deployment? What guarantees does it add?
3. Explain Services: ClusterIP vs NodePort vs LoadBalancer.
4. What is an Ingress and how does it differ from a Service of type LoadBalancer?
5. What are ConfigMaps and Secrets, and how are they surfaced to a Pod?
6. What does kubectl do under the hood when you run 'kubectl apply'?

INTERMEDIATE

1. How do readiness and liveness probes differ, and what happens when each fails?
2. Explain requests vs limits for CPU and memory. What is a Pod's QoS class and why does it matter at eviction time?
3. How does an Ingress Controller actually route traffic? What is the role of the controller vs the Ingress object?
4. How are Secrets stored in etcd by default, and how do you make them genuinely secure?
5. Walk through a rolling update and how you'd roll it back. What do maxSurge and maxUnavailable control?
6. How does service discovery and DNS work inside the cluster (CoreDNS)?

ADVANCED

1. A Pod is stuck in CrashLoopBackOff. Walk me through your diagnosis, step by step.
2. A node goes NotReady. What happens to its Pods, and what controls the timing?
3. How does the scheduler decide placement? Explain taints/tolerations, affinity, and topology spread.
4. How would you design multi-tenancy and isolation in a shared cluster (namespaces, RBAC, network policies, quotas)?
5. Explain how the HPA scales workloads and what can make it behave badly.
6. On EKS specifically: how do IRSA (IAM Roles for Service Accounts) and the AWS VPC CNI work?

DevOps, CI/CD & Automation

What they're testing: *Tests pipeline design, safe rollouts, and automation judgement (Jenkins, Helm, patching).*

BASIC

1. What problem does CI/CD solve? Define CI vs CD vs continuous deployment.
2. What is a Jenkins pipeline, and what's the difference between a scripted and a declarative pipeline?
3. What is a Jenkins agent/node, and why run builds on agents rather than the controller?
4. What is Helm, and what problem do Helm charts solve over raw manifests?
5. What are a Helm chart, a release, and values.yaml?
6. What is idempotency and why does it matter in automation scripts?

INTERMEDIATE

1. How do you manage secrets/credentials in a Jenkins pipeline safely?
2. Explain Helm templating: how do values flow into templates, and what does 'helm upgrade --install' do?
3. How would you structure a pipeline that builds, tests, scans, and deploys to multiple environments?
4. What is a Helm chart dependency, and how do you pin and manage versions?
5. How do you make a deployment safely reversible? Compare rollback strategies.
6. How do blue-green and canary deployments differ, and how would you implement each?

ADVANCED

1. Design a patching pipeline across EKS nodes, EC2 instances, and ElastiCache with minimal disruption. How do you sequence and validate it?
2. How do you drain and patch Kubernetes nodes without dropping traffic (cordon, drain, PDBs)?
3. How would you make pipelines reproducible and auditable across a whole org?
4. A Helm upgrade half-applied and left resources inconsistent. How do you recover?
5. How do you handle database or schema migrations safely within a deploy pipeline?
6. How do you prevent a bad change from rolling out everywhere at once (progressive delivery, gates, automated rollback on SLO breach)?

Observability & Root-Cause Analysis

What they're testing: *Tests whether you can instrument systems and actually find the cause when things break.*

BASIC

1. What are the three pillars of observability, and how do metrics, logs, and traces differ?
2. What does Prometheus do, and what is its data model (metric name + labels)?
3. What is the role of Grafana relative to Prometheus?
4. What is Kibana used for, and what sits behind it (the ELK/Elastic stack)?
5. What CloudWatch primitives exist (metrics, logs, alarms, dashboards) and how do they relate?
6. What is the difference between monitoring and observability?

INTERMEDIATE

1. How does Prometheus scrape targets, and how does service discovery find them?
2. Explain a PromQL rate() on a counter. Why can't you alert on a raw counter value?
3. What are the four golden signals, and why those four?
4. How do you design a good alert so it fires on symptoms, not noise? What is alert fatigue?
5. How would you ship application logs to Elasticsearch and make them searchable in Kibana?
6. What is the difference between CloudWatch Logs Insights and querying flow logs in Athena?

ADVANCED

1. Latency is up but CPU and memory look fine. Walk me through your RCA approach.
2. How do you correlate a spike across Prometheus, Grafana, Kibana, and CloudWatch to find a single root cause?
3. How would you design an RCA bot that ingests signals from multiple tools and proposes a cause? What are the failure modes?
4. What are RED and USE methods, and when do you reach for each?
5. How do you set meaningful SLOs and error budgets, and how do they drive on-call behaviour?
6. How do you avoid false root causes (correlation vs causation) when multiple things change at once?

Cloud Security & Compliance

What they're testing: *Tests CSPM, the frameworks named on the resume, and how you remediate safely.*

BASIC

1. What is Cloud Security Posture Management (CSPM), and what problem does it solve?
2. What are the CIS Benchmarks and what do they cover?
3. What is AWS Foundational Security Best Practices (FSBP)?
4. What is PCI-DSS and who must comply with it?
5. What is NIST (e.g. 800-53 / CSF) at a high level?
6. What is the shared responsibility model in AWS?

INTERMEDIATE

1. How does AWS Security Hub aggregate findings, and how do standards like CIS/FSBP map to controls?
2. What is the difference between a preventive, detective, and corrective control? Give an AWS example of each.
3. How does AWS Config evaluate compliance, and what is a Config rule vs a conformance pack?
4. How would you detect publicly exposed resources (S3 buckets, security groups open to 0.0.0.0/0)?
5. What is least privilege in practice, and how do you find and fix over-permissive IAM?
6. How do GuardDuty, Inspector, and Macie differ in what they detect?

ADVANCED

1. Design an agent that auto-remediates security findings. How do you make auto-remediation safe and reversible?
2. A finding says an S3 bucket is public but the business needs it public. How do you handle exceptions without weakening the baseline?
3. How do you remediate a finding at scale across hundreds of accounts without breaking workloads?
4. How do you prove continuous compliance to an auditor, and what evidence do you collect automatically?
5. What's your approach to prioritising findings (severity vs exploitability vs blast radius)?
6. How would you prevent remediation from causing an outage (guardrails, dry-run, staged rollout, rollback)?

AI / ML Engineering

What they're testing: *Tests the model lifecycle, CV/NLP fundamentals, and productionising models (Hugging Face).*

BASIC

1. Walk me through the end-to-end ML lifecycle from data to deployment.
2. What is the difference between training, validation, and test sets, and why keep them separate?
3. What is overfitting, how do you detect it, and how do you reduce it?
4. What is the difference between classification and regression? Give a metric for each.
5. What is a pre-trained model, and what is transfer learning / fine-tuning?
6. What is Hugging Face, and what do the Hub, Transformers, and Spaces each provide?

INTERMEDIATE

1. Explain precision, recall, F1, and when accuracy is misleading.
2. How do you handle class imbalance in a dataset?
3. What is data leakage, and how have you guarded against it?
4. How do you decide between a classical ML model and a deep-learning model for a task?
5. How do you deploy a model for inference, and what are the trade-offs of batch vs real-time serving?
6. What is an embedding, and where have you used one?

ADVANCED

1. How do you monitor a deployed model for drift and degradation, and what do you do when it drifts?
2. How do you make inference cheap and fast at scale (batching, quantisation, caching, hardware choice)?
3. How would you design a pipeline to retrain and safely roll out a new model version?
4. What are LLM agents, and how do they differ from a single prompt call? What are their failure modes?
5. How do you evaluate a generative model where there's no single correct answer?
6. How do you reason about bias, privacy, and PII when training on real-world data (e.g. faces, reviews)?

Python, Git & Linux

What they're testing: *Core engineering fundamentals that underpin every project.*

BASIC

1. What are Python lists vs tuples vs dicts vs sets, and when do you use each?
2. What is a virtual environment and why use one?
3. What is the difference between 'git merge' and 'git rebase'?
4. What do 'git fetch', 'git pull', and 'git push' each do?
5. What are Linux file permissions (rwx, chmod, chown), and how do you read 'ls -l' output?
6. What is the difference between a process and a thread?

INTERMEDIATE

1. What is the GIL in Python, and how does it affect threading vs multiprocessing?
2. Explain Python generators and when they save memory.
3. How do you resolve a merge conflict, and how do you undo a bad commit (revert vs reset)?
4. How do you find what's consuming CPU, memory, or disk on a Linux box (top, ps, df, du, iostat)?
5. What is a context manager ('with'), and why is it useful for files and connections?
6. How do you debug a hanging or slow Python program?

ADVANCED

1. How does asyncio work, and when is async the right tool vs threads vs processes?
2. A script works on your machine but fails on the server. Walk through isolating the cause (env, paths, permissions, versions).
3. How do you structure a Git branching strategy for a team shipping continuously?
4. How would you make a long-running job survive disconnects and resume after failure?
5. How do you profile and optimise a slow Python data-processing script?
6. How do you safely handle secrets in code and in a Git history that already leaked one?

Part 2 — Projects

Automated Patching Pipeline

BusinessNext · Jenkins, EKS, EC2, ElastiCache

What they're testing: *Be ready to defend sequencing, zero-downtime, validation, and rollback.*

BASIC

1. What problem does automated patching solve, and what were you patching (OS, nodes, cache engine)?
2. Why Jenkins for this, and what triggers the pipeline?
3. What does a patch cycle look like end to end in your design?
4. What is the difference between patching EC2, patching EKS worker nodes, and patching ElastiCache?
5. How do you know a patch succeeded?

INTERMEDIATE

1. How do you patch EKS nodes without dropping traffic? (cordon, drain, PodDisruptionBudgets)
2. How do you patch ElastiCache with minimal disruption, and what is the failover behaviour?
3. How do you stage patches across dev/stage/prod?
4. How do you handle a node that won't drain because of a stuck Pod?
5. How do you make the whole run idempotent and safely re-runnable?

ADVANCED

1. A patch passed in staging but broke production. How is your pipeline designed to catch and contain that?
2. How do you roll back a bad patch across the fleet quickly?
3. How do you prove to an auditor that every instance is patched to a required level?
4. How do you schedule patching around maintenance windows and avoid patching everything at once?
5. What metrics tell you the patching pipeline itself is healthy?

Security Compliance (Multi-Framework)

BusinessNext · CIS, AWS FSBP, PCI-DSS, NIST

What they're testing: *Expect framework specifics and how you turned findings into fixes.*

BASIC

1. Which frameworks did you enforce, and why those?
2. How did you scan AWS against these standards (Security Hub, Config, custom)?
3. What does a 'finding' look like, and what fields matter?
4. Give an example of a control from CIS and one from PCI-DSS you worked with.
5. What is a compliance baseline?

INTERMEDIATE

1. How do CIS and FSBP overlap, and how do you avoid double-counting findings?
2. How did you prioritise which findings to fix first?
3. How do you handle a control that the business legitimately needs to violate?
4. How do you track remediation over time and show the posture improving?
5. How do you separate detective from corrective work?

ADVANCED

1. How do you enforce these standards across many accounts consistently?
2. How do you prevent remediation from causing an outage?
3. How do you keep newly created resources compliant from day one (preventive guardrails)?
4. How do you produce audit evidence automatically?
5. Where do automated checks fall short, and how do you cover those gaps?

Guardex — CSPM Agent

BusinessNext · cloud security posture + auto-remediation

What they're testing: *This is your flagship; expect deep design questions and safety concerns.*

BASIC

1. What is Guardex and what problem does it solve?
2. What does 'automatically remediate findings' mean concretely? Give an example.
3. Where do the findings come from?
4. What does the agent do when it detects a misconfiguration?
5. How is it different from just running Security Hub?

INTERMEDIATE

1. How do you decide which findings are safe to auto-fix vs needing a human?
2. How do you make a remediation reversible?
3. How do you avoid a fix-loop where the agent and another system fight over a resource?
4. How do you test a remediation before it runs on production?
5. How do you handle permissions for the agent safely (least privilege, scoped roles)?

ADVANCED

1. Walk me through the architecture of Guardex end to end.
2. A remediation caused an outage. How is the system designed so that can't cascade?
3. How do you scale auto-remediation across hundreds of accounts without breaking workloads?
4. How do you prioritise and rate-limit remediations during a storm of findings?
5. How do you prove the agent's actions are auditable and compliant?
6. How would you add an approval workflow for high-risk fixes?

RCA Bot

BusinessNext · Prometheus, Grafana, Kibana, CloudWatch

What they're testing: *Expect correlation logic, data sources, and avoiding false causes.*

BASIC

1. What does the RCA bot do, and who uses it?
2. Which data sources does it pull from and why those four?
3. What is a 'root cause' vs a symptom in your design?
4. What does the output look like?
5. What triggers the bot to run?

INTERMEDIATE

1. How do you correlate a metric spike in Prometheus with log errors in Kibana?
2. How do you pull and align timestamps across tools with different clocks and resolutions?
3. How does the bot rank candidate causes?
4. How do you avoid confusing correlation with causation?
5. How do you keep it from producing noisy or confidently-wrong answers?

ADVANCED

1. Walk me through the architecture and the data flow of the RCA bot.
2. Two unrelated changes happened at the same time as an incident. How does the bot avoid a false conclusion?
3. How would you evaluate whether the bot's root-cause answers are actually correct over time?
4. How do you handle incidents the bot has never seen before?
5. How would you integrate the bot into the on-call/incident workflow safely?

Cost Optimization Tool

BusinessNext · AWS spend reduction

What they're testing: *Expect where waste hides and how you quantify savings safely.*

BASIC

1. What does the tool do, and what kinds of waste does it look for?
2. What are the biggest sources of unnecessary AWS spend?
3. How do you identify an underutilised resource?
4. What data does the tool read to make its decisions?
5. How do you measure the savings you achieved?

INTERMEDIATE

1. How do you distinguish 'idle' from 'low but needed' before recommending a change?
2. How do you right-size an instance safely without hurting performance?
3. What's the difference between On-Demand, Reserved, Savings Plans, and Spot, and when does each save money?
4. How do you find and clean up orphaned EBS volumes, snapshots, and idle Elastic IPs?
5. How do you attribute cost to teams or workloads (tagging, cost allocation)?

ADVANCED

1. How do you avoid a cost 'optimisation' that causes an outage or data loss?
2. How would you automate recommendations without auto-applying risky changes?
3. How do you reason about data-transfer and NAT costs, which are often hidden?
4. How do you prioritise savings by effort vs impact?
5. How do you keep savings from eroding over time (drift back to waste)?

End-to-End AWS Infra Deployment

BusinessNext · IAM, S3, EC2, SSM, EKS, VPC, VPN, ALB, ACM + K8s

What they're testing: *The broadest project; expect architecture and networking depth.*

BASIC

1. Walk me through delivering a client's infrastructure from scratch. What's the order?
2. How do you design the VPC (CIDR, subnets, route tables)?
3. How do you set up IAM for a new environment?
4. What's the role of ALB and ACM in the stack?
5. What Kubernetes resources did you deploy (StatefulSets, Deployments, Ingress, Secrets, ConfigMaps) and why?

INTERMEDIATE

1. How do you set up a site-to-site VPN tunnel to the client, and how do you verify it's up?
2. How do you expose a workload securely: Ingress, ALB, TLS via ACM, DNS?
3. How do you manage secrets and config for the workloads?
4. How do you provision and join EKS worker nodes?
5. How do you make the deployment repeatable (IaC, Helm) rather than click-ops?

ADVANCED

1. Design the whole thing for high availability and least privilege. Where are the single points of failure?
2. How do you structure networking for site-to-site VPN plus private egress plus internal routing?
3. How do you hand this off so the client can operate it safely?
4. How do you manage drift between the deployed state and the desired state?
5. If you had to rebuild the environment in a new region, how fast could you, and what makes that possible?

Kubernetes Diagnostic Agent

BusinessNext · AI-assisted cluster diagnostics

What they're testing: *Expect how it reads cluster state and turns it into plain-language guidance.*

BASIC

1. What does the agent do, and who is it for?
2. What cluster signals does it read (events, Pod status, logs, metrics)?
3. What does 'explains what went wrong in human-readable language' mean in practice?
4. Give an example problem it would catch and what it would say.
5. How does it access the cluster safely?

INTERMEDIATE

1. How do you map a raw symptom (CrashLoopBackOff, OOMKilled, pending Pod) to a clear explanation?
2. How do you keep its advice accurate rather than plausible-sounding but wrong?
3. How do you scope its permissions (read-only, RBAC)?
4. How do you handle clusters it hasn't seen before?
5. How does it decide the 'next action' to recommend?

ADVANCED

1. Walk me through the architecture from cluster event to recommendation.
2. How do you prevent it from recommending a dangerous action?
3. How would you evaluate whether its diagnoses are correct?
4. How would you extend it to act on the cluster, and what guardrails would that need?
5. How does it stay useful as Kubernetes versions and APIs change?

BI Platform Management

BusinessNext · Redash (Helm), Superset, Metabase

What they're testing: *Expect deployment, upgrades, access control, and sole-ownership questions.*

BASIC

1. What are Redash, Superset, and Metabase, and how do they differ?
2. What does it mean that you 'manage them for the whole organisation'?
3. What is Helm's role in deploying and upgrading Redash?
4. How do these tools connect to data sources?
5. What's a dashboard vs a query vs a data source in these tools?

INTERMEDIATE

1. Walk me through upgrading Redash via a Helm chart safely. How do you avoid breaking existing dashboards?
2. How did you implement Superset and Metabase end-to-end (deploy, auth, data connections)?
3. How do you manage user access and permissions across these tools for the whole org?
4. How do you back up dashboards, queries, and metadata?
5. How do you handle secrets (database credentials) for the data connections?

ADVANCED

1. A Helm upgrade of Redash failed midway. How do you recover without losing dashboards?
2. How do you scale these tools for many concurrent users org-wide?
3. How do you secure them (SSO, network exposure, row-level/data access control)?
4. How do you decide which tool to standardise on, or why run all three?
5. How do you monitor the health and query load of these BI platforms?

Face Recognition Model

Visacent · OpenCV, dlib, face_recognition (deployed on Hugging Face)

What they're testing: *Expect CV pipeline, accuracy, and privacy of biometric data.*

BASIC

1. Walk me through the face-recognition pipeline: detection, alignment, encoding, matching.
2. What do OpenCV, dlib, and the face_recognition library each contribute?
3. What is a face embedding, and how is a match decided?
4. How did you deploy it on Hugging Face?
5. What data did you train or evaluate on?

INTERMEDIATE

1. What's the difference between face detection and face recognition?
2. How do you handle lighting, pose, and occlusion?
3. What is your false-accept vs false-reject trade-off, and how do you tune the threshold?
4. How do you evaluate accuracy for a recognition system?
5. How do you speed up matching against many known faces?

ADVANCED

1. How do you handle the privacy and legal risk of storing biometric data?
2. How do you defend against spoofing (a photo of a face)?
3. How would you detect and reduce demographic bias in the model?
4. How do you keep the embedding database and threshold stable as you add people?
5. How would you productionise this for real-time use at scale?

Image Blur Model

Visacent · OpenCV, Pillow (deployed on Hugging Face)

What they're testing: *Expect detection-of-regions plus the blurring mechanics.*

BASIC

1. What does the model do, and what regions does it blur (faces, text, objects)?
2. What do OpenCV and Pillow each handle in the pipeline?
3. What blurring techniques exist (Gaussian, pixelation, median) and when do you use each?
4. How do you detect the region to blur before blurring it?
5. How did you deploy it?

INTERMEDIATE

1. How do you make the blur irreversible (so the original can't be recovered)?
2. How do you handle multiple regions, overlapping regions, or none found?
3. How do you balance blur strength vs the image staying useful?
4. How do you process large images efficiently?
5. How do you evaluate that the right things got blurred?

ADVANCED

1. How would you run this on video / streams rather than single images?
2. How do you handle false negatives (a sensitive region not detected)?
3. How would you scale this to batch-process millions of images cheaply?
4. What are the privacy guarantees, and how do you prove them?
5. How would you let a human review and correct the automated blur?

Embassy Scraper

Visacent · BeautifulSoup, Scrapy, Selenium (deployed on Hugging Face)

What they're testing: *Expect scraping strategy, anti-bot handling, and data structuring.*

BASIC

1. What does the scraper extract, and from what kind of pages?
2. When do you use BeautifulSoup vs Scrapy vs Selenium?
3. What's the difference between parsing static HTML and rendering a JavaScript page?
4. How do you structure the scraped data?
5. How did you deploy/run it?

INTERMEDIATE

1. How do you handle pagination, rate limits, and retries?
2. How do you deal with sites that block bots (headers, delays, rotating sessions)?
3. How do you make the scraper resilient when a site's layout changes?
4. How do you avoid scraping the same page twice and keep data fresh?
5. How do you validate and clean the extracted data?

ADVANCED

1. How would you scale scraping many sites concurrently without getting blocked?
2. How do you handle the legal and ethical side (robots.txt, terms of service, PII)?
3. How do you monitor and alert when a scraper silently breaks?

4. How would you design this to recover and resume after a crash mid-run?
5. How do you detect and handle CAPTCHAs or login walls?

Broken-Image Checker

Visacent · Requests, BeautifulSoup, Pillow (deployed on Hugging Face)

What they're testing: *Expect crawling, HTTP correctness, and image validation.*

BASIC

1. What does the tool do, and what counts as a 'broken' image?
2. What do Requests, BeautifulSoup, and Pillow each do here?
3. How do you find all images on a page?
4. How do you tell a broken image from a working one?
5. What's the output/report?

INTERMEDIATE

1. What HTTP status codes and errors indicate a broken image, and which are false alarms?
2. How do you tell a valid URL that returns a corrupt image (Pillow fails to open) from a 404?
3. How do you crawl a whole site, not just one page?
4. How do you handle redirects, lazy-loaded images, and relative URLs?
5. How do you make it fast over many pages (concurrency)?

ADVANCED

1. How would you run this continuously and alert on new breakages?
2. How do you avoid hammering a site while checking thousands of images?
3. How do you handle CDNs, authentication, and hotlink protection?
4. How would you scale this across many large sites?
5. How do you keep false positives low so people trust the report?

Reviews Analyzer

Visacent · NLTK, spaCy, Hugging Face Transformers

What they're testing: *Expect NLP pipeline, sentiment, and evaluation.*

BASIC

1. What does the analyzer extract from reviews (sentiment, topics, keywords)?
2. What do NLTK, spaCy, and Transformers each contribute?
3. What is tokenisation, and why does it matter?
4. What's the difference between rule-based and model-based sentiment analysis?
5. How did you deploy it?

INTERMEDIATE

1. How do you handle negation, sarcasm, and mixed sentiment in one review?
2. What is a transformer model, and why is it better than bag-of-words for this?
3. How do you extract topics or aspects (aspect-based sentiment)?
4. How do you preprocess messy, real-world review text?
5. How do you evaluate the quality of the sentiment output?

ADVANCED

1. How do you handle multiple languages in reviews?
2. How do you detect and handle fake or spam reviews?
3. How do you keep the model accurate as language and products change (drift)?
4. How would you serve this at scale and cheaply?
5. How do you turn raw sentiment into something the business can act on?

Test Case Automation

Cadence · Python, ~90% effort reduction

What they're testing: *Expect the problem, the mechanism, and how you proved the 90%.*

BASIC

1. What was the manual problem before your script?
2. What exactly did the Python script automate?
3. How did it keep test cases in sync with code changes?
4. How did you measure the ~90% reduction?
5. How did the team adopt it into their workflow?

INTERMEDIATE

1. How did the script detect what changed and which test cases needed updating?
2. How did you make sure it didn't silently corrupt or drop test cases?
3. How did you handle edge cases where the automation couldn't decide?
4. How did you test the automation itself?
5. What did you do when the script produced a wrong update?

ADVANCED

1. How would you generalise this so other teams could reuse it?
2. How did you make it safe to run repeatedly (idempotent)?
3. What were the limits of the automation, and where did a human still need to step in?
4. How would you integrate it into CI so it ran automatically on every change?
5. How did you build trust that the automated changes were correct?

Virtuoso UI Refactor

Cadence · UI refactor, added key buttons

What they're testing: *Expect the usability problem and how you scoped the change.*

BASIC

1. What is Cadence Virtuoso, and who uses it?
2. What was the usability problem you were solving?
3. What buttons/features did you add, and why those?
4. How did you decide what to change?
5. What was the before/after impact?

INTERMEDIATE

1. How did you refactor the UI without breaking existing workflows?
2. How did you gather what users actually needed?
3. How did you test the changes?
4. How did you handle backward compatibility?
5. What trade-offs did you make in the design?

ADVANCED

1. How did you ensure the refactor didn't introduce regressions?
2. How did you roll the change out to users safely?
3. How did you measure whether the change actually improved usability?
4. What would you do differently if you redid it?
5. How did you document and hand over the change?

Website Translation Tool

Sai Products · multilingual translation (deployed on Hugging Face)

What they're testing: *Expect the translation pipeline and how content is handled without duplication.*

BASIC

1. What does the tool do, and what problem does it solve?
2. How does it translate website content into any target language?
3. What does 'without duplicating content' mean in your design?
4. What models or services power the translation?
5. How did you deploy it on Hugging Face?

INTERMEDIATE

1. How do you extract translatable text from a page without breaking its structure/markup?
2. How do you handle things that shouldn't be translated (names, code, placeholders)?
3. How do you keep layout intact when translated text is longer or shorter?
4. How do you cache or reuse translations to avoid re-translating?
5. How do you evaluate translation quality?

ADVANCED

1. How would you scale this to large sites and many languages?
2. How do you handle right-to-left languages and locale-specific formatting?
3. How do you keep translations in sync when the source content changes?
4. How do you handle domain-specific terminology correctly?
5. What are the failure modes, and how does the tool degrade gracefully?